
Games Engine Design and Development Report

Jade(Yujie) Liang
5709365

University of Warwick

May 25, 2026

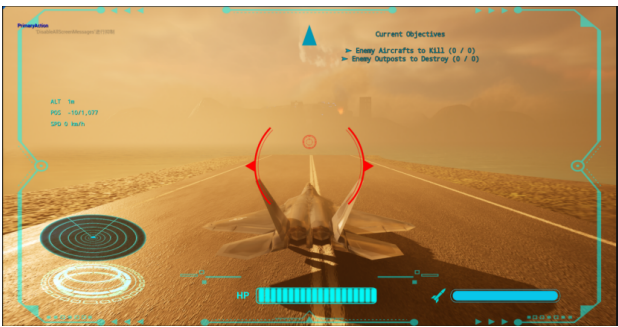


Figure 1: Final effect



Figure 2: Main HUD UI

1 Introduction

This game is a third-person air combat game where players need to pay attention to enemy planes in the sky, ground turrets, and the status of their own aircraft. Therefore, I implemented a real-time HUD subsystem to allow players to quickly read this information. The HUD system includes a beginning tutorial UI, a radar minimap, enemy indicators, an aircraft health bar, missile cooldown bar, target lock-on range indicator, crosshair, damage warning animation, aircraft status display, and multiple custom UI materials(see Figure 1).

2 Design Approach

The overall art style leans towards a futuristic, mechanical, and technological feel, so the UI's main color scheme is cool cyan, with a simple and easy-to-read style to avoid distracting the player.

I divided the UI into multiple widgets, which are

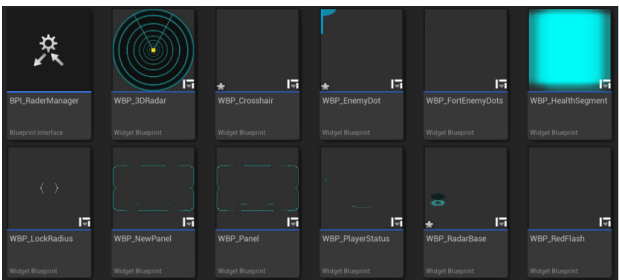


Figure 3: Main sub-widget UI

then attached to the Main HUD(see Figure 2 and Figure 3). Each widget contains smaller widget components; for example, the radar system is divided into *WBP_RadarBase* and *WBP_EnemyDots*. This is for easy expansion and convenient integration into the game; simply add the Main HUD to the Viewport in the *BP_Airplane*.

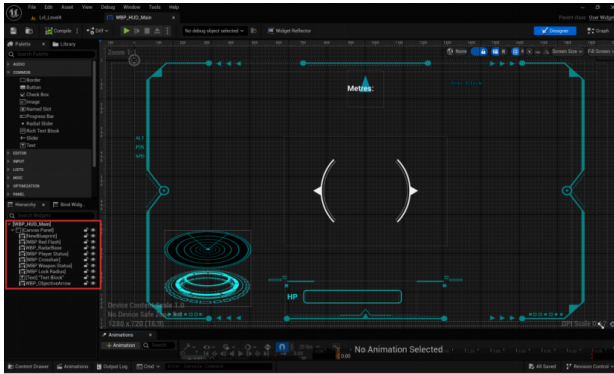


Figure 4: HUD Widget Structure

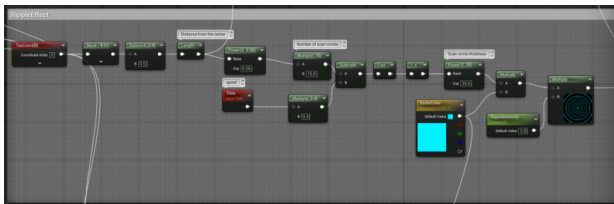


Figure 5: ripple effect material implementation

3 Technical details

3.1 HUD Widget Structure

see Figure 4

3.2 2D UMG Radar with Pseudo-3D Effect

The radar is implemented using a 2D screen-space UMG system, achieving a pseudo-3D effect. This approach was chosen because it maintains screen stability, avoiding issues such as camera roll, depth sorting, and inconsistent visibility. An elliptical layout, animated radar materials, directional arrows, and height bars work together to create a sense of depth while ensuring the radar is clear and easy to read.

3.2.1 Material part

The radar base uses a custom UI material instead of a static texture. The material consists of three visual layers: dynamic scanning ripples radiating outwards from the center, FOV guide lines indicating the direction of the field of view, and a fixed outer boundary. These three parts are combined in the material graph and applied to the UMG Image, giving the radar both a dynamic scanning effect and easy parameter adjustment. (see Figure 5, Figure 6 and Figure 7)

The final material effect is achieved by combining these three parts. (see Figure 8)

3.2.2 Functionality

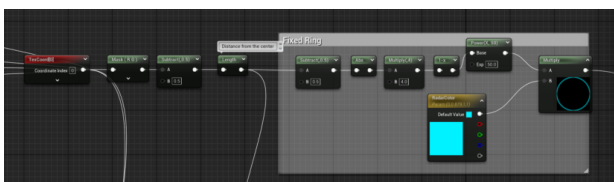


Figure 6: outer fixed ring effect material implementation

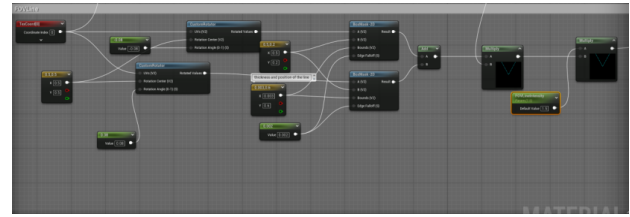


Figure 7: FOV line effect material implementation

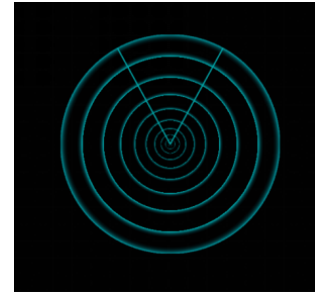


Figure 8: Final radar material

The system reads the enemy aircraft array passed in from outside, calculates the enemy's vector relative to the player, transforms it to the player's local space based on the player's orientation, then maps the enemy's 3D world location to 2D radar coordinates. Finally, it clamps the enemy points into the radar range based on the radar radius, and uses flatten Y to scale the radar vertically to achieve an elliptical effect, thus realizing a pseudo-3D effect.

Initially, during testing, an array of all enemies in the scene was obtained in the event construct (see Figure 9). However, the drawback of this approach was that once the project could dynamically spawn enemies, it was not possible to obtain the enemy and their position information in real time. Therefore, it was changed to call an externally provided enemy array interface to obtain the enemy list, with functions including adding and removing the array. (see Figure 10)

The radar does not create a new enemy indicator widget every frame. Instead, the EnemyDot widget is created only when the enemy list is received or updated. Within a tick cycle, the system only updates the position, direction, and altitude display of existing indicators. When an enemy is invalid or destroyed, the corresponding component is removed from the radar parent panel. (see Figure 11)

Then, the function to remove the node follows. If the Enemy has been destroyed or is not valid, the widget is removed from the parent panel. The enemy reference

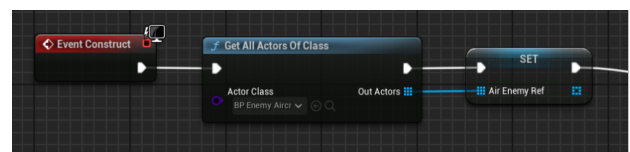


Figure 9: Previous method of reading enemy array

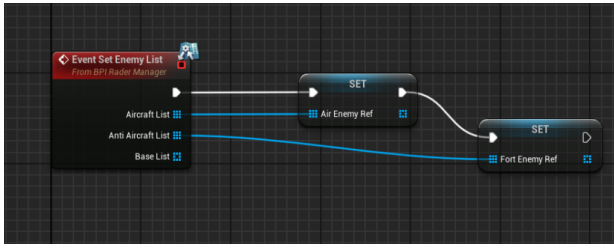


Figure 10: Later method of reading enemy array

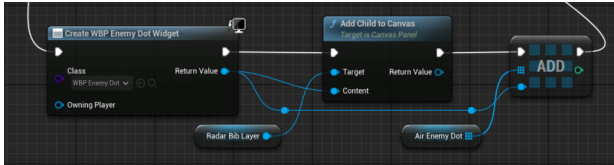


Figure 11: Methods of creating EnemyDot Widgets

itself is removed from the enemy list by the external enemy management logic, so the radar only needs to remove the related UI widget. (see Figure 12)

The enemy's position is converted from world space into the player's local space using the Inverse Transform Location node. This makes the radar direction depend on the player aircraft's orientation instead of the world axis. Because UMG screen coordinates use a downward-positive Y axis, the calculated radar Y value is inverted before being applied to the widget position. (see Figure 13)

The height difference between the enemy and the player is passed to the *UpdateHeightIndicator* function. This value controls the length and direction of the altitude bar and arrow in the EnemyDot widget. (see Figure 14)

In the *UpdateHeightIndicator* function, the variable *ParallelZoneHeight* is set to 5000 Units, approximately 50 meters, because Unreal Engine uses centimeters as the unit by default. If the absolute height difference between the enemy and the player is less than this threshold, the enemy will be considered to be at a similar height, and therefore the arrow and height bar will be hidden. In the radar, they are treated as parallel (the arrow and bar are hidden, leaving only the central enemy dot visible). (see Figure 15)

Regarding the vertical scaling of arrows and bars, it's worth noting that when the arrow extends upwards, the alignment should be set to (0.5, 1.0), and when it extends downwards, it should be set to (0.5, 0.0). (see Figure 16)

To apply a clamping effect to the coordinates, when

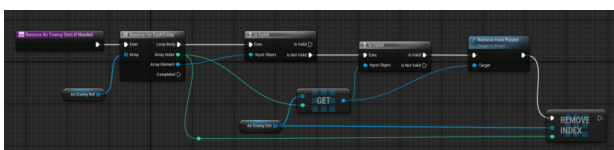


Figure 12: Methods of removing EnemyDot array index

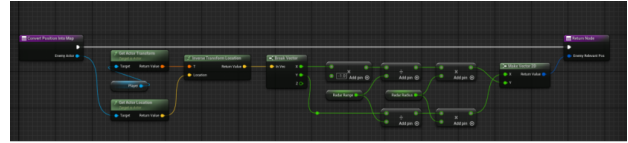


Figure 13: convert position into radar map

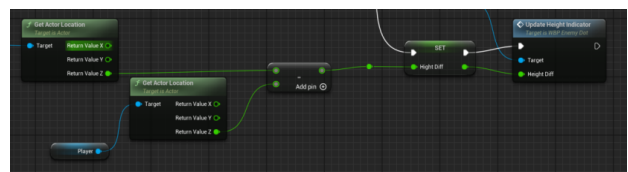


Figure 14: pass the HeightDiff to UpdateHeightIndicator function

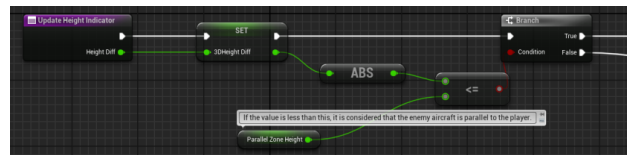


Figure 15: Parallel Height Threshold Setting

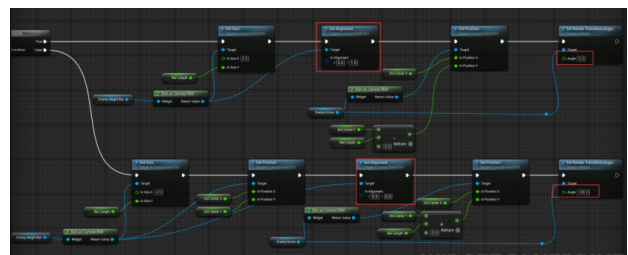


Figure 16: Arrow and bar settings

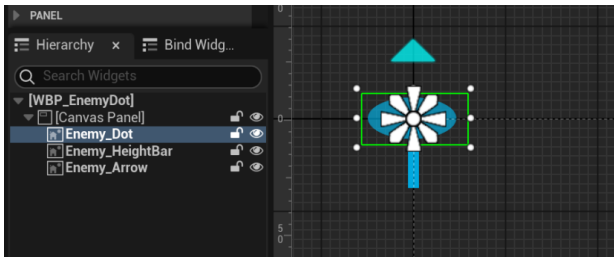


Figure 17: WBP_EnemyDots graph setting

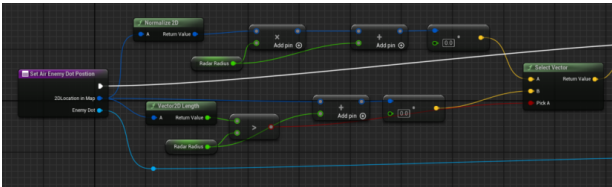


Figure 18: Method 1 for achieving the Clamp effect

the length of the Dots vector is greater than the Radar Radius, normalize the vector by taking only its direction and multiplying its length by a fixed radius value. This achieves the effect of clamping at the edge of the radar. Note that the entire enemyDots is composed of arrows and bars. To prevent the arrow and altitude bar from affecting the clamp calculation, the clamp calculation uses only the centre position of the enemy dot. (see Figure 17 and Figure 18)

To achieve the pseudo-3D elliptical effect, a *FlattenY* value smaller than 1 is applied to the radar's vertical coordinate. This compresses the radar display vertically and makes the circular radar appear as an ellipse, and the radar image is also vertically scaled to match this flattened coordinate system. (see Figure 19)

A UI material is applied to the arrow indicator, to keep its color and glow consistent with the radar style. (see Figure 20)

Final result: The runtime result shows three Enemy-Dot widgets on the radar. Their positions represent enemies above, below, and at a similar altitude to the player, demonstrating that the radar can display both horizontal direction and vertical height difference. The Elite Dangerous image is included only as a visual reference for the pseudo-3D radar style. (see Figure 21 and Figure 22)

3.3 Corsshair and the lock-on radius

The crosshair position is based on the existing aiming logic in *BP_Airplane*. In the aircraft blueprint, the aiming direction is represented by the end sphere of

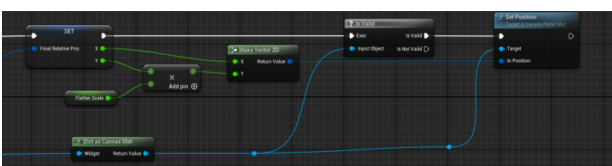


Figure 19: Method 2 for achieving the Clamp effect

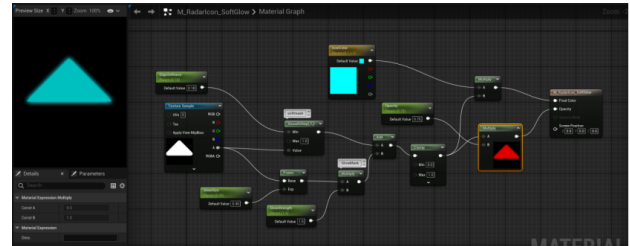


Figure 20: Methods for implementing arrow material

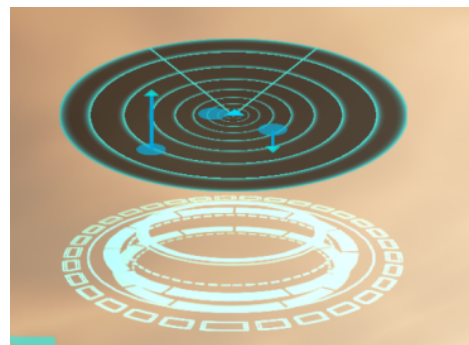


Figure 21: Final radar effect

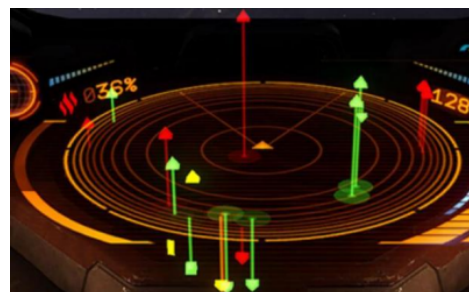


Figure 22: Elite Dangerous 3D radar

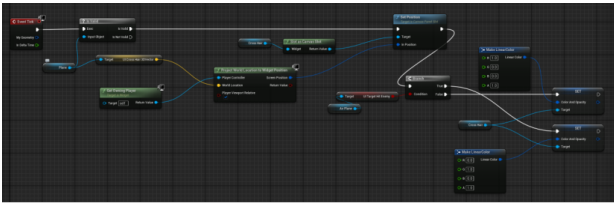


Figure 23: The crosshair implementation

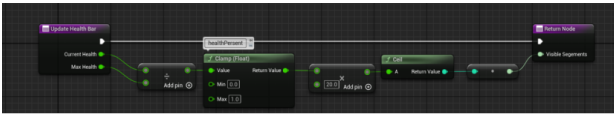


Figure 24: the health bar implementation

the TrackSphere. Instead of manually centering the crosshair on the screen, *WBP_Crosshair* reads the world coordinates of the end sphere and projects them into screen space using *Project World Location to Widget Position*. The position of the crosshair widget is then updated using the projected 2D position. This ensures that the UI crosshair aligns with the actual aiming direction used by the game logic.

The lock-on radius UI also uses the same screen-space projection method. It is displayed around the projected aiming point, showing the player the effective lock-on area. When tracking or overlap logic detects an Actor labeled *Enemy*, the crosshair or lock-on UI changes color to indicate that the target is within the firing or lock-on range.(see Figure 23)

3.4 Health Bar and Missile Cooldown Bar

The health bar has 20 slots. Health loss is represented by reducing the number of slots. For each segment, if its index is greater than or equal to the current visible segment count, it is hidden; otherwise, it remains visible.(see Figure 24)

The missile cooldown bar uses a percentage to represent the current cooldown progress. The cooldown value resets after the player fires a missile and recovers to full over time. This value is linked to the Progress Bar's Percent, indicating when the player can fire the next missile.

Final result:see Figure 25

3.5 Aircraft Status

The aircraft status consists of three text UI elements: speed (SPD), altitude (ALT), and position (POS, x/y).These values are read from the player aircraft and formatted using Format Text nodes before being displayed on the HUD.(see Figure 26)

Final result:see Figure 27

3.6 damage warning animation The damage warn-



Figure 25: the final effect of the health bar and missile cooldown bar

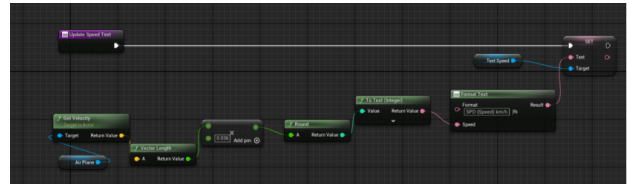


Figure 26: the implementation of showing the aircraft status

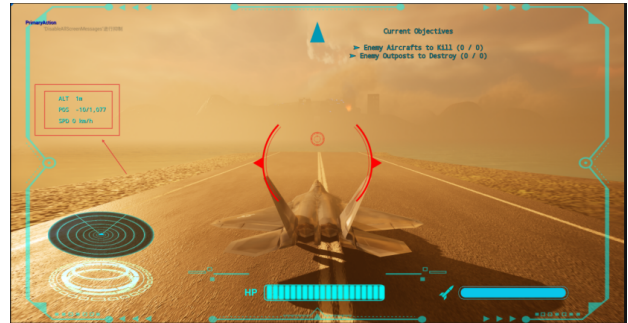


Figure 27: the final effect of the aircraft status UI

ing system compares the current aircraft health with the previous health value. When the plane's health is reduced, play a red Flash animation and the text animation "BEING ATTACKED!".(see Figure 28)

Two animation settings:see Figure 29

Final result:see Figure 30

3.7 Beginning tutorial UI

The beginning tutorial image was generated by ChatGPT and serves as the introductory guide interface. It explains basic controls, enemy types, combat techniques, and mission flow. This asset is used solely as a user interface guide image; The logic for showing and hiding the tutorial interface is implemented within the level blueprint.

Finally, add it to the level blueprint:(see Figure 31)
Final result:(see Figure 32)

4 Integration Details

The main process involves creating *WBP_HUD_Main* from *BP_Airplane* and then adding it to the Viewport. Sub-widgets are managed by the Main HUD. Data is primarily read from the Player aircraft reference, including position, health, and lock state. The enemy list is managed externally, mainly through *GameMode*, and passed to the radar through references or interfaces.

The UI does not alter the core combat logic; it only reads necessary gameplay state. Communication

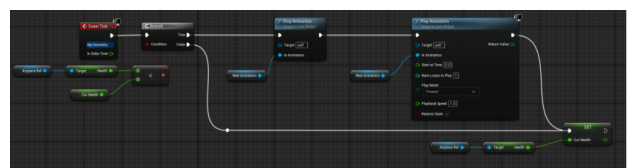


Figure 28: the warning animation implementation

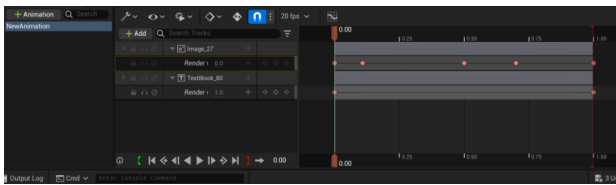


Figure 29: Damage Warning Animation Settings



Figure 30: the final effect of damage warning

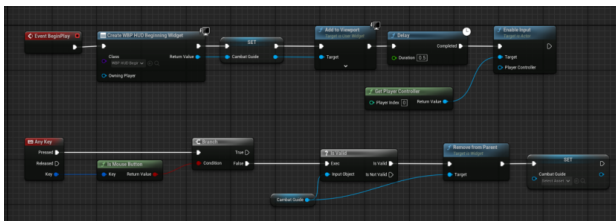


Figure 31: the implementation of showing the tutorial image



Figure 32: the tutorial image

is achieved through variables, references, and interfaces, reducing coupling between the UI and other blueprints. This makes the system more suitable for integration into team projects, as changes to the HUD are unlikely to affect aircraft movement, enemy behavior, or weapon logic.

5 Evaluation and Improvements

The HUD system provides the main information needed for the air combat gameplay. The radar helps the player understand enemy direction and relative altitude, while the crosshair and lock-on feedback connect the UI to the existing targeting logic. The health bar, missile cooldown bar, aircraft status display, and damage warning animation also provide immediate feedback during combat.

However, there are several limitations. First, some HUD elements are not fully responsive to different screen resolutions because the layout does not consistently use anchors, overlays, and size boxes. This may cause UI elements to shift on different displays. And some UI updates are based on Blueprint Tick, which may become less efficient if many enemies need to be tracked. Third, the current lock-on feedback mainly base on color changing, and could be improved with animation, sound feedback, and adding Enemy highlighted box.

6 Conclusion

In conclusion, my contribution was the implementation of a real-time HUD subsystem for the third-person air combat game. The system uses UMG widgets, Blueprint logic, and custom UI materials to provide radar, enemy indicators, lock-on feedback, health and cooldown bar, aircraft status information, damage warnings, and a beginning tutorial UI. The design focuses on readability and gameplay-driven feedback. It allows the UI to support the player during fast-paced combat and provides a functional HUD foundation for the final game.